AttritionIQ

Turnover Prediction as a Service

Technical Documentation - v1.0

Team 33 - Luka G., Hugo T., Taha I., Aurelien KJ., Roman S.
Capgemini x ESILV - TrustedAI Hackathon - March 2025

Table of Contents

Table of Contents 2
1. Project Overview 4
  1.1 Scope 4
  1.2 Technology Stack 4
2. System Architecture 5
  2.1 High-Level Architecture 5
  2.2 Data Flow 5
  2.3 Container Topology 5
3. Data Pipeline 7
  3.1 Source Dataset 7
  3.2 GDPR Preprocessing (backend/data/preprocessing.py) 7
    Step 1 - Direct PII Removal 7
    Step 2 - Protected Characteristics Exclusion (EU AI Act) 7
    Step 3 - ID Anonymization 7
    Step 4 - Feature Engineering 7
    Step 5 - Encoding and Imputation 8
  3.3 Output 8
4. Machine Learning Model 9
  4.1 Algorithm Selection 9
  4.2 Hyperparameters 9
  4.3 Validation 9
  4.4 Features 9
  4.5 SHAP Explainability (backend/model/explain.py) 10
    Background vector 10
    Output interpretation 10
5. API Reference 11
  5.1 Authentication 11
  5.2 Endpoints 11
    GET /health 11
    GET /employees 11
    GET /dashboard/stats 11
    GET /explain/{anon_id} 11
    POST /predict 11
    GET /metrics 12
6. Security Model 13
  6.1 Authentication and Authorization 13
  6.2 Input Validation 13
  6.3 CORS Policy 13

6.4 Data Isolation 13
6.5 Secret Management 13
7. Frontend 14
7.1 Stack 14
7.2 Pages 14
7.3 API Communication 14
8. Deployment 15
8.1 Prerequisites 15
8.2 Quick Start 15
8.3 Services 15
8.4 Stopping and Resetting 15
8.5 Directory Structure 15
9. Known Limitations 17
9.1 Model 17
9.2 Infrastructure 17
9.3 Responsible AI 17

1. Project Overview

AttritionIQ is a proof-of-concept SaaS platform for HR attrition prediction. It demonstrates the full pipeline from raw HR data to an explainable risk score, with GDPR compliance and responsible AI principles built in from the ground up.

This document covers the technical architecture, data pipeline, machine learning model, API endpoints, frontend, security model, and deployment instructions.

1.1 Scope

In scope for this prototype:

- GDPR-compliant data preprocessing pipeline
- Binary attrition classifier (Logistic Regression + SHAP explanations)
- FastAPI REST backend with authentication and input validation
- React frontend dashboard (risk table, SHAP waterfall, HR recommendations)
- Docker Compose local deployment

Out of scope for this prototype:

- Cloud or multi-tenant SaaS deployment
- Integration with HRIS systems
- Real-time data ingestion
- User management and SSO

1.2 Technology Stack

Layer	Technology	Version / Notes
ML / Data	scikit-learn	1.4.2 - Logistic Regression, StandardScaler, StratifiedKFold
Explainability	SHAP	0.45.0 - LinearExplainer for LR
Backend	FastAPI + Uvicorn	0.111.0 / 0.29.0 - async, OpenAPI auto-docs
Validation	Pydantic	2.7.1 - strict field-level validation
Frontend	React + Vite	18.2 / 5.2 - JSX, Recharts for visualizations
Containerization	Docker + Compose	Multi-service orchestration, named volumes
Data	pandas + numpy	2.2.2 / 1.26.4 - data pipeline

2. System Architecture

2.1 High-Level Architecture

The system is composed of three layers running in separate Docker containers orchestrated by Docker Compose. All processing is local - no data is sent to external services.

Layer	Description
Data	Raw CSV mounted read-only into the backend container. GDPR preprocessing runs at startup.
Backend	FastAPI (port 8000). On first launch: GDPR pipeline + model training in background thread. Serves REST API once ready.
Frontend	React + Vite dev server (port 3000). All API calls proxied through Vite to the backend container via Docker internal network.

2.2 Data Flow

The following sequence describes the lifecycle of a prediction request:

- 1. Raw CSV (HRDataset_v14.csv) is mounted into the backend container at /app/rawdata/
- 2. On startup, the GDPR preprocessing pipeline runs: PII dropped, IDs hashed, dates converted to numeric features, categoricals encoded.
- 3. The cleaned dataset is persisted to a named Docker volume at /app/artifacts/processed.csv
- 4. The Logistic Regression model is trained on the processed dataset. Artifacts (model.pkl, scaler.pkl, features.json, metrics.json) are persisted to the same volume.
- 5. The backend reports ready=true on the /health endpoint.
- 6. The React frontend polls /health until ready, then fetches dashboard stats and employee list.
- 7. When an HR user clicks 'Explain' on an employee, the frontend calls GET /explain/{anon_id}. The backend loads the processed row, runs SHAP LinearExplainer, and returns risk score + SHAP values + recommendations.

2.3 Container Topology

Docker Compose service definitions:

Service	Image	Port	Key Configuration
attritioniq-backend	python:3.11-slim	8000	ENV: API_KEY, DATA_RAW_PATH, ARTIFACTS_DIR. VOLUME: CSV (ro), model_artifacts (rw). HEALTHCHECK: python urllib every 15s.
attritioniq-frontend	node:20-alpine	3000	depends_on: backend (service_healthy). Vite proxy: /api/* -> http://attritioniq-backend:8000

3. Data Pipeline

3.1 Source Dataset

Dataset: IBM HR Analytics Employee Attrition (HRDataset_v14.csv)

- 311 employees, 36 raw columns
- Target variable: Termd (1 = left, 0 = active)
- Class distribution: ~67% active, ~33% terminated
- Source: Kaggle, CC0 license, synthetic/anonymized origin

3.2 GDPR Preprocessing (backend/data/preprocessing.py)

Step 1 - Direct PII Removal

The following columns are dropped before any ML processing:

Column	Reason for Removal
Employee_Name	Direct personal identifier
DOB	Date of birth - sensitive personal data (GDPR Art. 9)
Zip	Postal code - location quasi-identifier
State	US state - location quasi-identifier
ManagerName	Full name - indirect identifier

Step 2 - Protected Characteristics Exclusion (EU AI Act)

These columns are never used as model features, in compliance with anti-discrimination principles:

- Sex, GenderID - protected characteristic
- RaceDesc, HispanicLatino - protected characteristic
- MarriedID, MaritalStatusID, MaritalDesc - protected characteristic
- CitizenDesc - nationality, protected characteristic

Step 3 - ID Anonymization

EmpID is replaced by a 12-character prefix of its SHA-256 hash (anon_id). The original ID is discarded. The transformation is one-way and non-reversible without the original value.

```
anon_id = hashlib.sha256(str(emp_id).encode()).hexdigest()[:12]
```

Step 4 - Feature Engineering

Date columns are converted to numeric features:

- TenureDays: (DateofTermination or reference date) - DateofHire
- DaysSinceLastReview: reference_date - LastPerformanceReview_Date

Raw date columns are then dropped.

Step 5 - Encoding and Imputation

- Remaining string columns (Department, Position, RecruitmentSource) are label-encoded
- Missing values are filled with column median

3.3 Output

The processed dataset (processed.csv) contains 13 features + 1 target column (Termd) + 1 identifier (anon_id). No PII is present.

4. Machine Learning Model

4.1 Algorithm Selection

Algorithm: Logistic Regression (scikit-learn LogisticRegression)

Logistic Regression was selected over XGBoost or Random Forest for three reasons specific to this dataset:

Reason	Detail
Small dataset	With 311 rows, tree-based models memorize training examples and produce overconfident 0/1 predictions. Logistic Regression generalizes more reliably.
Probability calibration	LR produces naturally calibrated probabilities. A score of 60% reflects genuine ~60% attrition likelihood - essential for decision support.
SHAP consistency	SHAP LinearExplainer produces mathematically exact explanations for linear models - the feature contributions are directly interpretable as log-odds shifts.

4.2 Hyperparameters

Parameter	Value	Rationale
C (inverse regularization)	0.1	Moderate L2 regularization - reduces overfitting on small dataset
class_weight	balanced	Reweights the loss function proportional to class frequency - handles the 67/33 imbalance softly
solver	lbfgs	L-BFGS optimizer - stable for L2-penalized LR with small to medium datasets
max_iter	1000	Ensures convergence on all feature configurations
Feature scaling	StandardScaler	Zero mean, unit variance - required for LR to weight features fairly

4.3 Validation

Model performance is evaluated with 5-fold stratified cross-validation (StratifiedKFold, random_state=42). Stratification preserves the class distribution in each fold.

Metrics computed per fold: AUC-ROC, F1 Score, Accuracy. Final metrics are the mean across folds.

4.4 Features

Feature	Type	Range	Description
EngagementSurvey	Float	0-5	Annual engagement score
EmpSatisfaction	Int	1-5	Self-reported satisfaction
Absences	Int	0-n	Absences in review period
Salary	Int	> 0	Annual compensation (USD)
DaysLateLast30	Int	0-30	Lateness incidents last 30 days
SpecialProjectsCount	Int	0-n	Projects beyond standard scope
PerfScoreID	Int	1-4	Performance rating (1=PIP, 4=Exceeds)
EmpStatusID	Int	1-5	Employment status code
Department (encoded)	Int	0-n	Label-encoded department
Position (encoded)	Int	0-n	Label-encoded job title
RecruitmentSource (encoded)	Int	0-n	Recruitment channel
TenureDays (derived)	Int	> 0	Days from hire to reference date
DaysSinceLastReview (derived)	Int	> 0	Days since last performance review

4.5 SHAP Explainability (backend/model/explain.py)

SHAP (SHapley Additive exPlanations) provides per-prediction feature contribution scores. The implementation uses SHAP LinearExplainer, which is exact and fast for linear models.

Background vector

The background (baseline) is a zero vector in the StandardScaled feature space. This corresponds to the average employee in the original feature space, because StandardScaler centers data at zero. This makes the expected_value the model probability on the 'average' employee, always in [0,1].

Output interpretation

- risk_score: calibrated attrition probability [0,1] from the Logistic Regression model
- expected_value: model probability on the average employee (baseline, ~0.30 for this dataset)
- shap_values: per-feature contribution to the log-odds of attrition, sorted by absolute value
- top_risk_factors: features with positive SHAP values (push toward attrition)
- top_protective_factors: features with negative SHAP values (push away from attrition)

5. API Reference

5.1 Authentication

All endpoints (except /health) require an X-API-Key header. The key is set via the API_KEY environment variable.

```
X-API-Key: dev-secret-change-in-prod
```

In production, replace with a strong random secret and inject via the .env file (never committed to Git).

5.2 Endpoints

GET /health

Returns backend readiness status. Polled by the frontend until ready=true.

Response: { status: 'starting|preprocessing|training|ready|error', ready: bool, error: str|null }

GET /employees

Returns the full anonymized employee list with risk scores, sorted by risk descending.

Query: no parameters. Response: { employees: [...], total: int }

Each employee object: { anon_id, risk_score, risk_label, department, position, engagement, satisfaction, absences, actually_left }

GET /dashboard/stats

Aggregated analytics for the main dashboard.

Response includes: total_employees, attrition_rate, avg_engagement, avg_satisfaction, avg_absences, high_risk_count, medium_risk_count, low_risk_count, dept_attrition[], feature_importance[], model_metrics

GET /explain/{anon_id}

Full SHAP explanation for a specific anonymized employee.

Response: { risk_score, prediction, expected_value, shap_values[], top_risk_factors[], top_protective_factors[], recommendations[] }

POST /predict

Predicts attrition risk for a manually provided feature vector. Used for what-if simulations.

Body (JSON):

Field	Type / Range	Notes
EngagementSurvey	float 0-5	Required
EmpSatisfaction	int 1-5	Required
Absences	int 0-100	Required
Salary	int 0-500000	Required
DaysLateLast30	int 0-30	Required
SpecialProjectsCount	int 0-50	Required

GET /metrics

Returns model performance metrics and bias audit results from metrics.json.

Response: { model_type, auc_roc, f1, accuracy, brier_score, prob_distribution, bias_audit }

6. Security Model

6.1 Authentication and Authorization

- All endpoints protected by X-API-Key header validation (FastAPI Security dependency)
- Key configured via environment variable API_KEY - never hardcoded or committed
- Key stored in .env file, which is listed in .gitignore
- .env.example provides a template with placeholder value only

6.2 Input Validation

All API inputs are validated by Pydantic models before reaching business logic:

- Strict typing: int, float, str with explicit constraints
- Range validation: ge/le constraints on all numeric fields
- No raw string inputs accepted - all features are typed numerics
- PII rejection: the /predict endpoint accepts only numeric feature vectors, never names or identifiers

6.3 CORS Policy

CORS is configured permissively for local development (allow_origins=['*']). In a production deployment, this must be restricted to the known frontend origin.

6.4 Data Isolation

- Raw CSV is mounted read-only into the backend container
- Processed data and model artifacts are stored in a named Docker volume, not the host filesystem
- No external API calls are made during preprocessing, training, or inference
- No PII is transmitted to the frontend - only anonymized IDs and numeric features

6.5 Secret Management

Secret	Management
API_KEY	Environment variable, .env file (gitignored), injected at Docker Compose runtime
Dataset (CSV)	Local file, excluded from Git via .gitignore, mounted read-only into container
Model artifacts	Stored in Docker named volume, not persisted to host filesystem or version control

7. Frontend

7.1 Stack

- React 18 + Vite 5 - component-based SPA
- Recharts - data visualization (pie chart, bar charts, waterfall SHAP chart)
- Vite proxy - all /api/* requests proxied to the backend container

7.2 Pages

Component	Description
Dashboard.jsx	Main landing page. KPI cards (total employees, attrition rate, avg engagement, high risk count, AUC-ROC). Risk distribution pie chart. Global SHAP feature importance bar chart. Attrition by department bar chart. Model performance metrics. Polls /health until backend ready.
EmployeeTable.jsx	Full employee list with risk score progress bars, risk level badges, and filter buttons (All / High / Medium / Low). Search by anonymized ID. Clicking 'Explain' navigates to ExplainView.
ExplainView.jsx	Per-employee SHAP explanation. Risk score hero (with baseline delta). Horizontal SHAP waterfall chart (red = risk, green = protective). Top risk factors and protective factors lists. Recommended HR actions (priority-coded). Human-in-the-loop disclaimer.

7.3 API Communication

All API calls are centralized in frontend/src/api.js. The base URL is /api, which Vite proxies to the backend container on http://attritioniq-backend:8000. The API key is hardcoded to the dev default value - this must be replaced by an environment variable injection mechanism before production use.

8. Deployment

8.1 Prerequisites

- Docker Desktop (with Docker Compose v2+)
- WSL2 (Windows) or native Linux/macOS for running start.sh
- The dataset file: data/HRDataset_v14.csv

8.2 Quick Start

```
git clone <repo-url>
cd attrition-fixed
cp data/HRDataset_v14.csv ./data/
sed -i 's/\r//' start.sh && chmod +x start.sh
./start.sh
```

On first launch, the backend will:

- Build Docker images (~3-4 minutes on first run, cached on subsequent runs)
- Run GDPR preprocessing (~2 seconds)
- Train the Logistic Regression model (~3 seconds)
- Serve the API on port 8000

The frontend starts once the backend healthcheck passes (~60-90 seconds total).

8.3 Services

- Frontend: http://localhost:3000
- API (Swagger docs): http://localhost:8000/docs
- Health check: http://localhost:8000/health

8.4 Stopping and Resetting

```
docker-compose down          # stop containers, keep volumes
docker-compose down -v       # stop containers AND delete model artifacts (forces
retraining)
```

8.5 Directory Structure

Path	Purpose
backend/data/preprocessing.py	GDPR anonymization pipeline
backend/model/train.py	Model training + bias audit + artifact serialization
backend/model/explain.py	SHAP LinearExplainer + HR recommendations
backend/main.py	FastAPI application + startup pipeline thread

Path	Purpose
frontend/src/pages/	React pages: Dashboard, EmployeeTable, ExplainView
frontend/src/api.js	Centralized API client (all backend calls)
docker-compose.yml	Service orchestration + volume definitions
data/	Place HRDataset_v14.csv here (gitignored)
docs/model_card.md	Model documentation (purpose, metrics, risks)
docs/data_card.md	Dataset documentation (GDPR, features, limitations)

9. Known Limitations

9.1 Model

- Small dataset (311 rows): results are indicative, not production-grade. Retraining on a real organization's HR data is required before any operational use.
- US-centric source: labor market dynamics, legal context, and cultural factors differ in EU environments.
- Mixed termination reasons: the Termd flag does not distinguish voluntary resignations from involuntary terminations, introducing label noise for a retention-focused use case.
- No temporal dimension: the model has no knowledge of trends over time.

9.2 Infrastructure

- Local deployment only: this prototype is not designed for multi-tenant or cloud SaaS deployment.
- CORS policy: must be restricted before any internet-facing deployment.
- API key: the dev default must be replaced before any non-local use.
- Frontend API key: the API key in api.js is not injected via environment variable - this must be addressed for production.

9.3 Responsible AI

- Human-in-the-loop: the system surfaces a disclaimer on every prediction screen. No employment decision should be made on model output alone without HR professional review.
- Bias audit: audited across PerfScoreID subgroups only. A full production audit would require evaluation across additional demographic dimensions.